

Московский авиационный институт
(Национальный исследовательский университет)
Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №5
по курсу «Теория сложности алгоритмов»
Вариант №2

Выполнила студентка: Бондарева Е. Е.

Группа: М8О-109СВ-25

Преподаватель: Рассказова В. А.

Дата: ____05.05.2026____

Оценка: _____

Москва, 2026

Задание

Для заданного языка

1. построить описание верификатора с полиномиальной временной сложностью и соответствующего сертификата принадлежности;
2. реализовать данный верификатор в виде программы;
3. провести тестовые исследования, демонстрирующие совпадение фактической временной сложности с теоретической.

$\text{HALF} - \text{CLIQUE} = \{ \langle G \rangle : G - \text{неориентированный граф с } m \text{ вершинами, который содержит клику размером не менее } \frac{m}{2} \}$;

Теоретическая часть

Через HAMPATH обозначим задачу поиска гамильтонова пути в ориентированном графе, т.е. такого пути, который проходит через вершины графа ровно один раз:

$\text{HAMPATH} = \{ \langle G, s, t \rangle : \text{в графе } G \text{ есть гамильтонов путь из } s \text{ в } t \}$

Опр. 5.1. Верификатором языка A называется ДМТ V такая, что $w \in A$ тогда и только тогда, когда существует $s \in \Sigma^*$ для которой V допускает w, s . Строка s при этом называется сертификатом.

Замечание 5.1. Иными словами, верификатор -- это алгоритм проверки кандидата s в решение задачи w на предмет того, что s действительно является решением. Т.е. верификатор -- это алгоритм, который подразумевает наличие сертификата (возможный кандидат в решение). При этом сам поиск сертификата s остается за пределами верификатора. Таким

образом, результатом работы верификатора является ответ Да если s является решением для w , и Нет если s НЕ является решением для w .

Опр. 5.2. Классом NP (называется класс всех языков, верифицируемых за полиномиальное время на детерминированной МТ).

Замечание 5.2. Аббревиатура NP происходит от сокращения Nondeterministic Polynomial, что означает существование недетерминированной МТ, решающей язык A . Иными словами, для всех языков в классе NP существует недетерминированный полиномиальный алгоритм решения и детерминированный полиномиальный алгоритм проверки (верификатор). Еще более бытовой подход к определению класса NP связан с наличием для задачи алгоритма полного перебора. Таким образом, разрешимые по Тьюрингу языки и определяют класс NP.

Теорема 5.1. Язык $A \in NP$ тогда и только тогда, когда существует НМТ, которая решает A за полином.

Доказательство. Пусть $A \in NP$. Тогда по определению существует полиномиальный верификатор V . Предположим, что V решает A за время n^k . Тогда НМТ: N = на входе w : недетерминированно записать на ленту произвольную строку s длины не более n^k ; запустить V на $\langle w, s \rangle$, затем ответить то же самое, что V .

Опр. 5.3. Пусть $t: \mathbb{N} \rightarrow [0, +\infty)$. Тогда $NTIME\ t(n) = A \subset \Sigma^*$: существует НМТ, которая решает A за время $O(t(n))$.

Замечание 5.3. Таким образом, теорема 5.1.утверждает, что

$$NP = \bigcup_{k=1}^{\infty} NTIME(n^k)$$

Замечание 5.4. Класс NP весьма обширный. Гамильтонов путь, клика, независимое множество, сумма подмножеств и т.д. — это задачи, в которых может быть реализован полный перебор, но для которых не установлена принадлежность классу P. Все они при этом лежат в NP.

Код программы

```
import time
import random

def verify_half_clique(graph_matrix, certificate):
    """
    Полиномиальный верификатор для языка HALF-CLIQUE.
    :param graph_matrix: Матрица смежности графа G (размер m x m)
    :param certificate: Список вершин (сертификат)
    :return: True, если сертификат доказывает принадлежность, иначе
    False
    """
    m = len(graph_matrix)

    # 1. Проверка размера сертификата (не менее m/2)
    if len(certificate) < m / 2:
        return False

    # 2. Проверка корректности вершин в сертификате (нет ли дубликатов
или неверных индексов)
    # 2.1. Проверка дубликатов
    if len(set(certificate)) != len(certificate):
        return False

    # 2.2. Проверка индексов
    for v in certificate:
        if v < 0 or v >= m:
```

```

        return False

    # 3. Проверка того, что все вершины в сертификате попарно
    # соединены (образуют клику)
    cert_size = len(certificate)
    for i in range(cert_size):
        u = certificate[i]
        for j in range(i + 1, cert_size):
            v = certificate[j]
            # Если между какой-то парой вершин нет ребра - это не
клик
            if graph_matrix[u][v] == 0:
                return False

    return True

def generate_test_data(m, k):
    """
    Генерирует граф размера m и заведомо правильный сертификат (клику
    размера m/2)
    для тестирования наихудшего (самого долгого) случая работы
    верификатора.
    """
    # Инициализация пустой матрицы смежности
    graph = [[0] * m for _ in range(m)]

    # Случайным образом выбираем вершины, которые станут кликой
    certificate = random.sample(range(m), k)

    # Соединяем все вершины сертификата между собой (создаем клику)
    for i in range(len(certificate)):
        for j in range(i + 1, len(certificate)):
            u = certificate[i]
            v = certificate[j]
            graph[u][v] = 1
            graph[v][u] = 1

    return graph, certificate

def sample_test(graph, cert):
    is_valid = verify_half_clique(graph, cert)
    print(f"Матрица смежности:")
    for row in graph:
        print(f"\t", *row)
    print(f"Сертификат: \n\t", *cert)
    print(f"{'Валидный' if is_valid else 'Не валидный'} сертификат для
данного графа")
    print()

```

```

def custom_tests():
    graph = [
        [0, 1, 1, 0],
        [1, 0, 1, 0],
        [1, 1, 0, 0],
        [0, 0, 0, 0],
    ]
    cert = [0, 1, 2]
    sample_test(graph, cert)

    graph = [
        [0, 1, 1, 0],
        [1, 0, 1, 0],
        [1, 1, 0, 0],
        [0, 0, 0, 0],
    ]
    cert = [0, 1, 3]
    sample_test(graph, cert)

def run_experiments():
    """
    Функция для эмпирической проверки временной сложности  $O(m^2)$ .
    """
    # Будем удваивать количество вершин графа
    sizes = [100, 200, 400, 800, 1600, 3200, 6400]
    prev_time = None

    print(f"{'Размер графа (m)':<20} | {'Размер серт-та':<15} | {'Время (сек)':<15} | {'Отношение t(m)/t(m/2)':<20}")
    print("-" * 75)

    for m in sizes:
        k = (m + 1) // 2 # Размер клики

        graph, cert = generate_test_data(m, k)

        # Замеряем только время работы самого верификатора
        start_time = time.perf_counter()
        is_valid = verify_half_clique(graph, cert)
        end_time = time.perf_counter()

        # Удостоверимся, что верификатор отработал корректно
        assert is_valid == True

        t = end_time - start_time

        # Вычисляем во сколько раз увеличилось время при увеличении m
        # в 2 раза
        ratio = t / prev_time if prev_time else 0.0

```

```

        prev_time = t

        print(f"{m:<20} | {len(cert):<15} | {t:<15.6f} | {ratio:<20.2f}")

    print('\n')

    custom_tests()

if __name__ == "__main__":
    run_experiments()

```

Вывод программы:

Размер графа (m)	Размер серт-та	Время (сек)	Отношение $t(m)/t(m/2)$
100	50	0.000062	0.00
200	100	0.000222	3.59
400	200	0.001139	5.13
800	400	0.006204	5.44
1600	800	0.033255	5.36
3200	1600	0.088381	2.66
6400	3200	0.593199	6.71

Матрица смежности:

```

0 1 1 0
1 0 1 0
1 1 0 0
0 0 0 0

```

Сертификат:

```

0 1 2

```

Валидный сертификат для данного графа

Матрица смежности:

```

0 1 1 0
1 0 1 0
1 1 0 0
0 0 0 0

```

Сертификат:

```

0 1 3

```

Не валидный сертификат для данного графа

Вывод

В результате выполнения лабораторной работы для заданного языка было построено описание верификатора с полиномиальной временной сложностью и соответствующего сертификата принадлежности, реализован данный верификатор в виде программы, проведены тестовые исследования, демонстрирующие совпадение фактической временной сложности с теоретической при условии, что $\text{HALF} - \text{CLIQUE} = \{\langle G \rangle: G - \text{неориентированный граф с } m \text{ вершинами, который содержит клику размером не менее } \frac{m}{2}\}.$